SMARTBRIDGE SKILL WALLET PROJECT DOCUMENTATION

Pocket Smart AI: Your Smart Budget & Recommendation Assistant

Team ID: SWTID-2026-5883

TEAM WORK ALLOCATION & MILESTONE RESPONSIBILITIES

TEAM MEMBER	ASSIGNED ROLE	ASSIGNED WORK & MILESTONE RESPONSIBILITIES
1. Jashvini.K	Lead AI & System Architect	**Milestone 1: Model Selection & Architecture** • Configure API Keys & Secure Storage (`.env` setup) • Model Research & Evaluation (LLAMA 3.3-70B Versatile) • Define 3-Tier System Architecture & Data Flow
2. Indhumathi.D	Backend Developer	**Milestone 2: Core Functionalities Development** • Develop Budget Calculation & Recommendation Modules • Implement Flask Backend Routing & Request Handling • Input Validation & Error Handling (Max 2000 chars)
3. Jayapriya Rajkumar	AI Integration & Prompt Specialist	**Milestone 3: App.py Core Logic & Parsing** • Implement Main Logic in `app.py` • Engineer Dynamic Prompts (Tone & Category Rules) • Develop Robust JSON Parser (`extract_json`)
4. Jonal.A	Frontend Developer	**Milestone 4: User Interface & UI/UX Design** • Responsive Dark Glassmorphism UI (HTML5/CSS3) • Interactive Form & Output Display Panels • Asynchronous Fetch API & Loader Animations
5. Kavibharathi.K	DevOps & QA Engineer	**Milestone 5: Public Deployment & Testing** • Local Virtual Environment & Dependency Setup • Ngrok Public Tunneling (`run_public.py`) • End-to-End System Testing & Documentation

PROJECT OVERVIEW & DESCRIPTION

Pocket Smart AI is an intelligent financial planning and personal budgeting assistant powered by Generative AI. Designed to give users instant, personalized financial guidance and smart expenditure recommendations, the application simplifies complex budget calculations into structured, actionable advice.

Utilizing high-speed LLM inference via the **Groq API** using the `llama-3.3-70b-versatile` model, Pocket Smart AI evaluates income, expense patterns, savings goals, and spending habits. Built using the **Flask** web application framework and styled with a dark glassmorphism user interface, the system ensures seamless user interaction and public accessibility via **Ngrok** secure tunneling.

CORE AI MODULES & CAPABILITIES

1. Smart Budget Breakdown Generator

Generates tailored financial allocations using the standard *50/30/20* rule (*Needs = 0.50 × Income*, *Wants = 0.30 × Income*, *Savings = 0.20 × Income*) adjusted dynamically according to user priorities and financial constraints.

2. Personalized Recommendation Engine

Provides ten tailored expense-reduction recommendations, highlighting priority cost-cutting areas and intelligent substitute suggestions.

3. Actionable Financial Call-to-Actions (CTAs)

Delivers three concise, urgency-driven action steps tailored to immediate user goals (e.g., debt reduction, emergency fund setup, automated savings).

APPLICATION SCENARIOS

Scenario 1: Student / Entry-Level Budgeting (Strict Savings)

A user enters a monthly income of $1,500 with high housing costs. Pocket Smart AI provides a strict budget allocation, suggests 10 cost-effective lifestyle adjustments, and outlines 3 immediate saving action steps in seconds.

Scenario 2: Professional Debt Consolidation (Aggressive Paydown)

A mid-level professional seeks to pay down credit card debt while saving for travel. The AI adjusts allocation ratios, prioritizes high-interest debt reduction, and provides structured recommendation cards.

Scenario 3: Freelance / Variable Income Planning

A freelancer inputs average monthly earnings and safety margin preferences. The system generates flexible spending buffers and dynamic savings recommendations.

TECHNICAL ARCHITECTURE & DATA FLOW

Pocket Smart AI follows a modular, responsive 3-tier system architecture:

[User Interface (HTML/CSS/JS)] ──(AJAX Request)──> [Flask Backend (app.py)]

↓ (Structured Prompt & Rules)

[Groq API (LLAMA 3.3-70B Model)] ──(JSON Response)──> [Parsed Output Display]

- **Frontend:** Responsive HTML5/CSS3 glassmorphism UI with vanilla JavaScript for asynchronous fetch requests.
- **Backend Engine:** Flask web application handling request routing, payload validation, prompt assembly, and JSON extraction.
- **AI Integration:** Groq Inference Client running `llama-3.3-70b-versatile` with dynamic temperature tuning ($T = 0.8$).
- **Security & Deployment:** Secure environment management via `.env` and public tunnel exposure using `Ngrok`.

DETAILED MILESTONE EXECUTION WORKFLOW

Milestone 1: Model Selection & Architecture (Assigned to Jashvini.K)

- **Activity 1.1: API Key Provisioning:** Registered Groq Console account, generated API key, configured local `.env` file (`GROQ_API_KEY=...`), and set `.gitignore` protection.
- **Activity 1.2: Generative Model Selection:** Evaluated Groq LLM suite for low-latency financial text generation. Selected `llama-3.3-70b-versatile` for high structural compliance and speed.
- **Activity 1.3: Architectural Specification:** Designed system sequence and component interaction diagrams connecting UI, Flask routes, and Groq endpoints.
- **Activity 1.4: Project Environment Setup:** Established directory hierarchy (`static/`, `templates/`, `app.py`, `requirements.txt`).

Milestone 2: Core Functionalities Development (Assigned to Indhumathi.D)

- **Activity 2.1: Feature Specification:** Built backend logic for Budget Breakdown, Recommendation Engine, and Financial CTA Generator.
- **Activity 2.2: Flask API & Validation:** Configured `/generate` POST route, implemented payload sanitization, and enforced input length constraints ($L \leq 2000$ characters).

Milestone 3: App.py Core Integration (Assigned to Jayapriya Rajkumar)

- **Activity 3.1: Application Orchestration:** Developed main application logic in `app.py` with endpoints:
  - `/` : Renders primary web portal interface.
  - `/generate` : Processes user input, calls Groq API, extracts structured JSON, and returns results.
- **Activity 3.2: Robust JSON Extractor:** Built `extract_json()` helper supporting multi-tier regex parsing to guaranteed valid JSON output from LLM responses.

Milestone 4: Frontend Development (Assigned to Jonal.A)

- **Activity 4.1: Glassmorphism Web Interface:** Designed single-page web app (`index.html`, `style.css`) featuring background blur effects, animated gradient blobs, and responsive form components.
- **Activity 4.2: Dynamic UI Rendering:** Implemented `main.js` with dynamic card generation, clipboard functionality, and auto-scroll animations.

Milestone 5: Deployment & Public Exposure (Assigned to Kavibharathi.K)

- **Activity 5.1: Local Testing & Server Setup:** Executed virtual environment activation and Flask development deployment on port 5050.
- **Activity 5.2: Public Deployment with Ngrok:** Configured `run_public.py` using `pyngrok` to expose local server securely to public HTTPS endpoints.

CORE SERVER IMPLEMENTATION (APP.PY SNIPPET)

```
from flask import Flask, render_template, request, jsonify
from groq import Groq
import os, json, re
from dotenv import load_dotenv

load_dotenv()
app = Flask(__name__)
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    user_info = data.get("product_info", "").strip()
    if not user_info:
        return jsonify({"error": "Financial query required."}), 400

    prompt = f"Analyze budget and recommend savings for: {user_info}"
```

```
    completion = client.chat.completions.create(
        messages=[{"role": "user", "content": prompt}],
        model="llama-3.3-70b-versatile",
        temperature=0.8
    )
    return jsonify({"success": True, "result": completion.choices[0].message.content})
```

CONCLUSION & FUTURE ROADMAP

Pocket Smart AI provides an efficient, AI-driven personal finance assistant that seamlessly bridge user inputs with intelligent financial recommendations. Future expansions include OCR receipt scanning, multi-currency support, and bank integration APIs for real-time tracking.